

Intent-Spezifikation — ReqPOOL Software Estimation Manager

Version 2.0 · Status: bereit zur agentischen Umsetzung · Einheit: PT = Personentage (1 PT = 8 h) · Bezeichner in JSON/Code englisch

1 · Zweck & Kern-Intent

Das Werkzeug wandelt objektive Größenparameter eines Softwarevorhabens in eine parametrisierte, nachvollziehbare Aufwandsschätzung (PT, inkl. Bandbreite) und eine daraus abgeleitete Scope-Beschreibung. Verbindlich sind die Kontrakte (Abschnitte 5, 6) und die Testfälle (Abschnitt 7); die technische Umsetzung ist frei wählbar.

- **Parametriert & kalibrierbar:** Alle Treiber, Faktoren, Aufschläge, Caps und der Sockelaufwand liegen in einer externen, *versionierten* Kalibrierung — im Code keine fest verdrahteten Schätzwerte.
- **Deterministisch & nachvollziehbar:** Gleiche Eingabe + gleiche Kalibrierung → identische Ausgabe; jede Zahl ist auf „Eingabe × Kalibrierwert“ rückführbar. Die Ausgabe enthält den vollständigen Rechenweg und die Kalibrierungsversion.
- **Anschlussfähig:** JSON-Ausgabe zur direkten Weiterverarbeitung durch das nachgelagerte Angebots-/Honorarwerkzeug.

Änderungen ggü. v1.0: Rundungsregeln verbindlich definiert (goldener Testfall korrigiert: Bandbreite **431**/538/700 statt fälschlich 430/538/700) · Komplexitätsklassen je Treiber-Instanz · Caps für Modifikatoren · Sockelaufwand (baseEffort) · Risikoprofil (riskLevel) · Kalibrierungs-Versionierung & Eingabe-Echo · Fehlerkontrakt. Mit den Default-Werten reproduziert v2.0 die v1.0-Ergebnisse exakt (einzige Abweichung: korrigierte Rundung der Bandbreite).

2 · Eingabeparameter

2.1 Size Driver (additiv), Typ Count

Parameter	Bedeutung
pages	Oberflächen / Seiten (UI-Screens)
useCases	fachliche Use Cases / Abläufe
businessObjects	Geschäftsobjekte (Entitäten)
interfaces	externe Schnittstellen / Integrationen
batches	Batch-/Hintergrundverarbeitungen

Typ Count — zwei zulässige Formen: (1) einfach: $\text{int} \geq 0$, alle Instanzen gelten als medium (wie v1.0); (2) klassifiziert: $\{ \text{"simple": } \text{int} \geq 0, \text{"medium": } \text{int} \geq 0, \text{"complex": } \text{int} \geq 0 \}$, fehlende Schlüssel = 0.

2.2 Modifikatoren (multiplikativ)

Parameter	Bereich	Bedeutung
languages	$\text{int} \geq 1$	Sprachen → Lokalisierungsfaktor (gecappt)
roles	$\text{int} \geq 1$	Benutzerrollen → Berechtigungsfaktor (gecappt)
users	$\text{int} \geq 0$	Nutzeranzahl → Skalierungsfaktor (gestuft, auf Gesamtsumme)

2.3 Kontextparameter (optional)

Parameter	Werte	Default
riskLevel	low medium high	medium
outputLanguage	de en	de

Erweiterbar: Weitere Treiber, Modifikatoren, Komplexitätsklassen und Skalierungsstufen lassen sich allein über die Kalibrierung ergänzen, ohne Änderung der Berechnungslogik.

3 · Berechnungslogik (deterministisch, in dieser Reihenfolge)

$E[d]$	$= \text{count}[d] \times \text{rate}[d] \times 1,0$	# einfache Form (int) – Klasse medium
$E[d]$	$= \sum_k (\text{count}[d][k] \times \text{rate}[d] \times \text{classFactor}[k])$	# klassifizierte Form, $k \in \{ \text{simple}, \text{medium}, \text{complex} \}$
$f_language$	$= \min(1 + (\text{languages} - 1) \times \text{language.increment}, \text{language.cap})$	
f_role	$= \min(1 + (\text{roles} - 1) \times \text{role.increment}, \text{role.cap})$	
f_scale	$= \text{Faktor der ersten Nutzer-Stufe mit } \text{maxUsers} \geq \text{users}$	(letzte Stufe: $\text{maxUsers} = \text{null} = \text{unbegrenzt}$)
$E_adj[d]$	$= E[d] \times \prod \{ f_m \mid d \in \text{modifier}[m].\text{appliesTo} \}$	
RealizationCore	$= \text{baseEffort} + \sum E_adj[d]$	
Realization	$= \text{RealizationCore} \times f_scale$	# $\text{scale.appliesTo} = \text{"total"}$
Subtotal	$= \text{Realization} \times (1 + \sum \text{Phasenaufschläge})$	
Total	$= \text{Subtotal} \times (1 + \text{risk}[\text{riskLevel}])$	
low / high	$= \text{Total} \times (1 + \text{range.low}) / \text{Total} \times (1 + \text{range.high})$	

Rundungsregeln (verbindlich): (1) Alle Zwischenwerte werden ungerundet weitergerechnet (IEEE-754 double genügt; Rundung nur bei Ausweisung). (2) low, totalRounded, high werden aus dem *ungerundeten* Total berechnet und kaufmännisch auf ganze PT gerundet (**round half up**, kein Banker's Rounding). (3) Übrige PT-Werte: Ausweis mit 2 Nachkommastellen (half up). (4) Stets gilt $\text{range.expected} = \text{totalRounded}$.

4 · Kalibrierung — Default-Startwerte (vor Produktiveinsatz zu kalibrieren)

Size Driver		rate (PT/Einheit)	
pages		2,0	
useCases		3,0	
businessObjects		1,5	
interfaces		5,0	
batches		3,0	
Komplexitätsklasse		classFactor	
simple		0,7	
medium		1,0	
complex		1,5	
Modifikator	increment	cap	appliesTo
language	0,08	1,60	pages , useCases
role	0,05	1,50	pages , useCases

Skalierungsstufe (users)		f_scale
≤ 50		1,00
≤ 500		1,05
≤ 5 000		1,10
≤ 50 000		1,20
unbegrenzt (null)		1,30
Phasenaufschlag (% v. Realization)		Wert
requirements 15 % · design 10 % · test 20 %		Σ 73 %
deployment 8 % · projectManagement 15 % · documentation 5 %		
Risikoreserve (riskLevel)		Bandbreite
low 10 % · medium 15 % · high 25 %		range.low –20 % · range.high +30 %

Sockelaufwand: baseEffort = 0 PT (Default für v1.0-Kompatibilität; für den Produktiveinsatz 5–15 PT empfohlen — Projektsetup, CI/CD, Basis-Deployment).

Hinweise: Werte sind plausible Startwerte und mit ReqPOOL-Erfahrungswerten (historische Projekte) zu kalibrieren. Phasenaufschläge sind lineare Zuschläge auf die Realisierung (bewusste Vereinfachung). Die Nutzer-Stufen springen an den Grenzen (50 → 51: +5 %) — beabsichtigt, den Anwendern zu kommunizieren.

4.1 Kalibrierungs-Schema (JSON, versioniert)

```
{ "version": "2.0.0", "baseEffort": 0,
  "drivers": { "pages": { "rate": 2.0 }, "useCases": { "rate": 3.0 }, "businessObjects": { "rate": 1.5 },
    "interfaces": { "rate": 5.0 }, "batches": { "rate": 3.0 } },
  "complexityClasses": { "simple": 0.7, "medium": 1.0, "complex": 1.5 },
  "modifiers": {
    "language": { "increment": 0.08, "cap": 1.6, "appliesTo": [ "pages", "useCases" ] },
    "role": { "increment": 0.05, "cap": 1.5, "appliesTo": [ "pages", "useCases" ] },
    "scale": { "appliesTo": "total", "tiers": [ { "maxUsers": 50, "factor": 1.00 }, { "maxUsers": 500, "factor": 1.05 },
      { "maxUsers": 5000, "factor": 1.10 }, { "maxUsers": 50000, "factor": 1.20 }, { "maxUsers": null, "factor":
1.30 } ] } },
  "phases": { "requirements": 0.15, "design": 0.10, "test": 0.20, "deployment": 0.08, "projectManagement": 0.15,
"documentation": 0.05 },
  "risk": { "low": 0.10, "medium": 0.15, "high": 0.25 }, "range": { "low": -0.20, "high": 0.30 },
  "rounding": { "mode": "half-up", "totalsDecimals": 0, "intermediateDecimals": 2 } }
```

5 · Ein-/Ausgabe-Kontrakt (JSON)

Eingabe

```
{ "outputLanguage": "de",
  "riskLevel": "medium",
  "parameters": {
    "pages": 20,
    "useCases": { "simple": 10,
      "medium": 15, "complex": 5 },
    "businessObjects": 15,
    "interfaces": 4, "batches": 5,
    "languages": 3, "roles": 6,
    "users": 2000 } }
```

Fehlerkontrakt

```
{ "errors": [ { "path": "parameters.languages",
  "code": "MIN_VALUE",
  "message": "languages must be ≥ 1" } ] }
```

Ausgabe (Erfolgsfall)

```
{ "meta": { "specVersion": "2.0", "calibrationVersion":
"2.0.0",
  "input": { ... Echo der validierten Eingabe inkl.
Defaults ... } },
  "estimate": {
    "unit": "PT", "baseEffort": 0,
    "driverEfforts": { "pages": { "raw": 40.0, "adjusted":
58.0 }, ... },
    "appliedModifiers": { "language": 1.16, "role": 1.25,
"scale": 1.10 },
    "realizationCore": 246.0, "realization": 270.6,
    "phases": { "requirements": 40.59, ... }, "subtotal":
468.14,
    "riskLevel": "medium", "riskContingency": 70.22,
    "total": 538.36, "totalRounded": 538,
    "range": { "low": 431, "expected": 538, "high":
700 } },
    "scope": { "summary", "breakdown[]", "inScope[]",
"assumptions[]", "exclusions[]" } }
```

Pflicht-Validierungen (keine stillen Defaults): Typen/Wertebereiche gemäß Abschnitt 2; unbekannte Schlüssel in parameters → UNKNOWN_PARAMETER; unbekannte Klassen-Schlüssel → Fehler; Kalibrierung wird beim Laden gegen Schema 4.1 validiert (Stufen aufsteigend, letzte Stufe null, Caps ≥ 1). Bei Schema-Verletzung wird keine Schätzung ausgegeben.

6 · Scope-Beschreibung (regelbasiert erzeugt)

summary (Template, DE-Default): „Die Anwendung umfasst {pages} Oberflächen in {languages} Sprache(n), realisiert {useCases} Use Cases auf {businessObjects} Geschäftsobjekten, bindet {interfaces} Schnittstelle(n) an, stellt {batches} Batch-Verarbeitung(en) bereit und ist rollenbasiert für {roles} Rolle(n) sowie {users} Nutzer ausgelegt.“ Bei klassifizierten Counts wird die Gesamtsumme eingesetzt; die Klassenverteilung erscheint im breakdown.

inScope / assumptions / exclusions werden je Parameter erzeugt (leere Treiber entfallen). Standard-Ausschlüsse: Betrieb/Hosting & Infrastruktur, Lizenz-/Drittsoftware, fachliche Datenmigration (falls nicht als Schnittstelle/Batch modelliert), Hardware, Schulung über Doku hinaus, Wartung nach Go-Live. Bei outputLanguage: "en" analoge englische Templates; Zahlen bleiben unverändert.

7 · Testfälle (verbindliche Abnahmeanker)

7.1 Goldener Testfall (identisch zu v1.0, Rundung korrigiert)

Eingabe: pages 20 · useCases 30 · businessObjects 15 · interfaces 4 · batches 5 · languages 3 · roles 6 · users 2000 · riskLevel medium (Default) · Kalibrierung = Defaults aus Abschnitt 4. — **Faktoren:** f_language = 1,16 · f_role = 1,25 · f_scale = 1,10

Größe	Wert (PT)
driverEfforts adjusted: pages 58,0 · useCases 130,5 · businessObjects 22,5 · interfaces 20,0 · batches 15,0	—
RealizationCore	246,00
Realization (× f_scale 1,10)	270,60
Subtotal (+ 73 % Phasen)	468,14
Total (+ 15 % Risiko) → gerundet	538,36 → 538
Bandbreite (low / expected / high)	431 / 538 / 700

Rechenweg Bandbreite: Total ungerundet 538,3587 → low = 430,687 → 431; high = 699,866 → 700. (v1.0 wies fälschlich 430 aus — Folge der fehlenden Rundungsregel.)

7.2 – 7.5 Ergänzende Testfälle

#	Eingabe	Erwartung
7.2 Komplexitätsklassen	pages: {simple:5, medium:10, complex:5}, übrige Treiber 0, languages 1, roles 1, users 0	$E[\text{pages}] = (5 \times 0,7 + 10 \times 1,0 + 5 \times 1,5) \times 2,0 = \mathbf{42,00 \text{ PT}}$; Total = $42 \times 1,73 \times 1,15 = 83,56 \rightarrow \mathbf{84 \text{ PT}}$; Bandbreite 67 / 84 / 109
7.3 Cap-Wirkung	wie 7.1, aber languages = 10	ungecappt f = 1,72 → Cap greift: f_language = 1,60
7.4 Stufengrenzen	users = 50 / 51 / 5 000 / 5 001	f_scale = 1,00 / 1,05 / 1,10 / 1,20
7.5 Degenerierter Fall	alle Size Driver 0, languages 1, roles 1, users 0	Total = baseEffort × 1,73 × 1,15 (Default: 0 PT); gültige Ausgabe; Hinweis in assumptions, dass keine Größentreiber erfasst wurden

8 · Nicht-funktionale Intents & Definition of Done

Empfohlene Umsetzung (nicht bindend): library-first, TypeScript/Node (Zod, Vitest) oder gleichwertig Python. Reihenfolge: Schemata & Default-Kalibrierung → Kern/Formel → Scope-Generator → CLI/Ausgabe → Testfälle 7.1–7.5 grün stellen → optional UI/Lokalisierung.

- ✓ Eingabe **und Kalibrierung** gegen Schema validiert; klare Validierungsfehler gemäß Fehlerkontrakt statt stiller Defaults.
- ✓ Berechnung nutzt ausschließlich die externe Kalibrierung (keine Magic Numbers); meta.calibrationVersion wird ausgewiesen.
- ✓ Ausgabe ist deterministisch, enthält den vollständigen Rechenweg und das Eingabe-Echo; validiert gegen das Ausgabe-Schema.
- ✓ Rundung exakt nach Abschnitt 3 (round half up; Bandbreite aus ungerundetem Total).
- ✓ Bandbreite low/expected/high sowie Scope inkl. exclusions werden ausgewiesen; Kern seiteneffektfrei als estimate(input, config) → output, CLI/UI optional darum herum.
- ✓ Testfälle 7.1–7.5 als automatisierte Tests grün; README mit Aufruf-Beispiel (JSON ein → JSON aus).